

Diet Problem Formulation

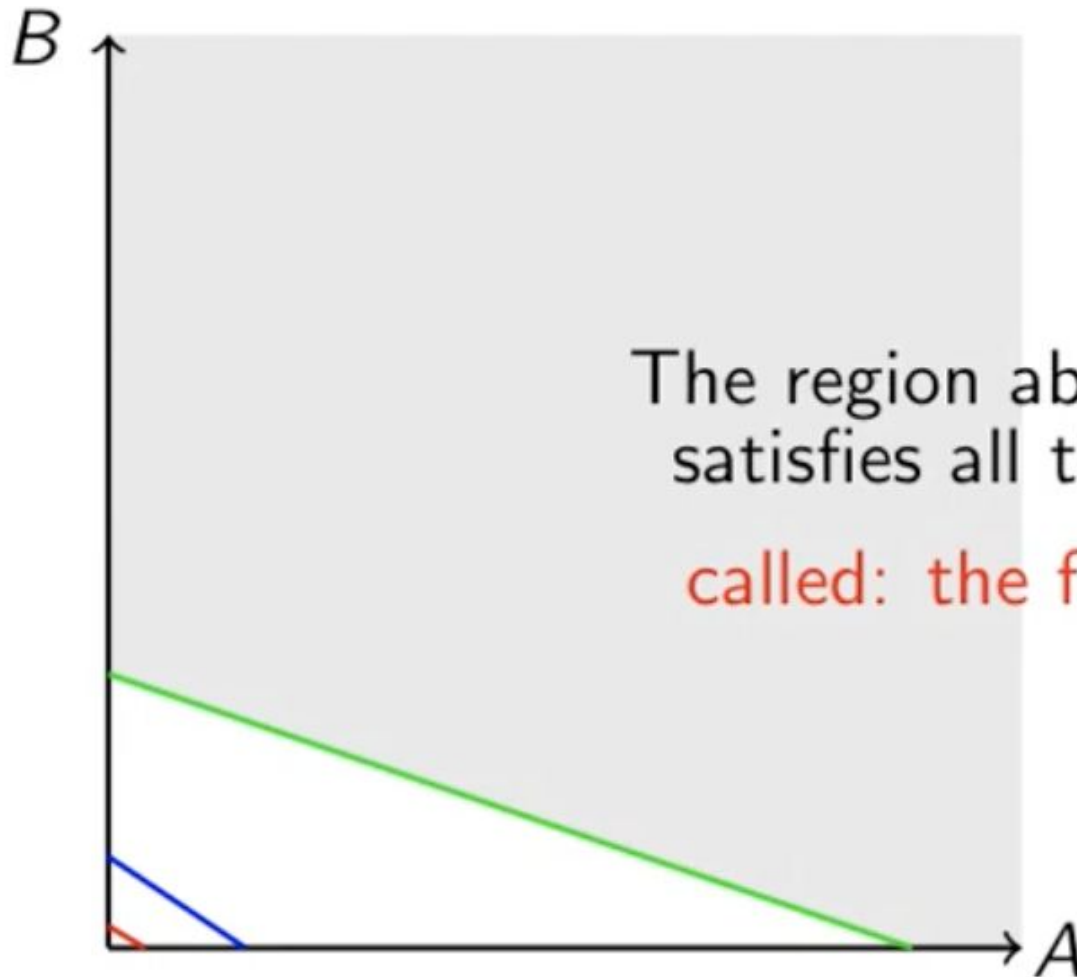
Food	Protein (g/unit)	Vit C (mg/unit)	Iron (mg/unit)	price (cent/unit)
Apple	0.4	6	0.4	8
Bananas	1.2	10	0.6	10
minimum	70	50	12	

Variables: A , B

Cost function: $f(A, B) = 8A + 10B$

Constraints:

$$\begin{cases} 0.4A + 1.2B \geq 70 & (C1) \\ 6A + 10B \geq 50 & (C2) \\ 0.4A + 0.6B \geq 12 & (C3) \\ A, B \geq 0 & (C4) \end{cases}$$



Two Linear Programming Applications

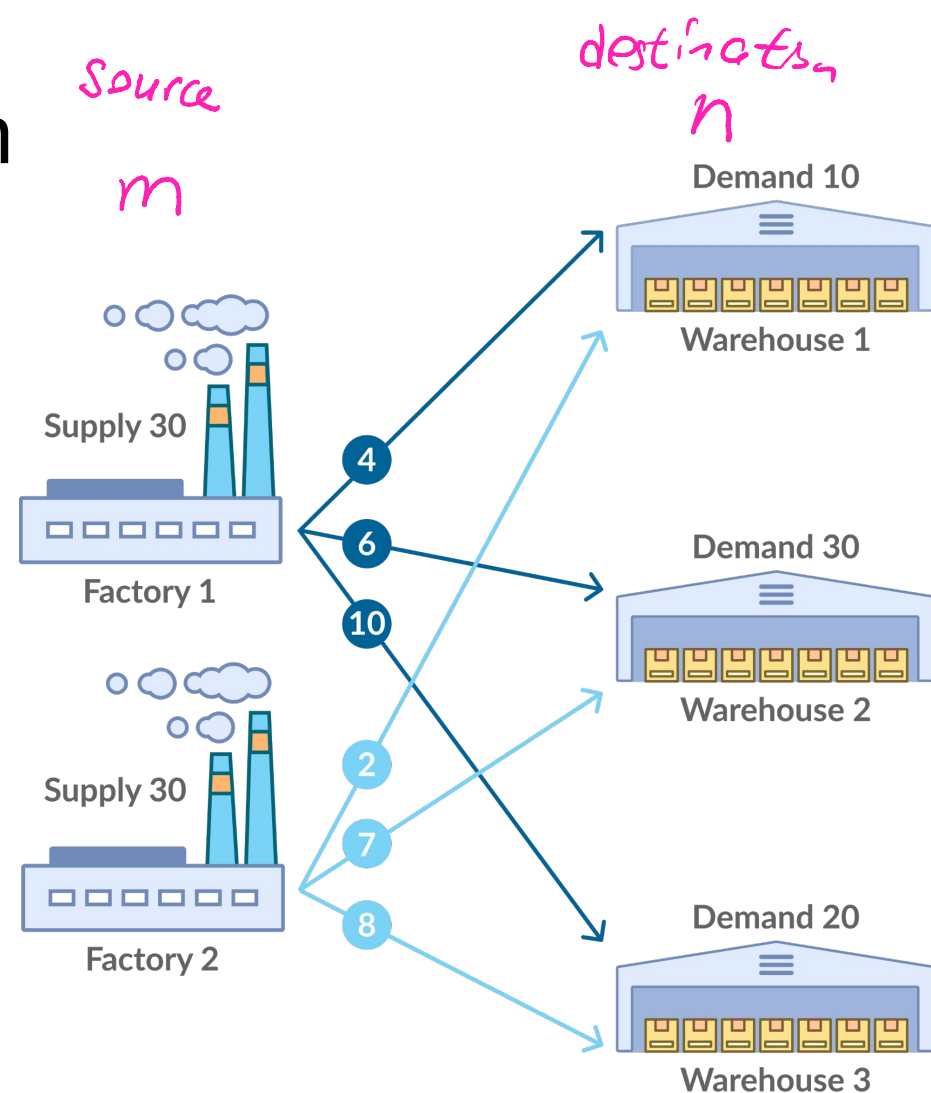
Two special LP problems

- Two special linear programming problems that have been applied in many real life situations are:
 - **Transportation Problems**
 - **Assignment Problems**
- They usually require a very large number of constraints and variables
- However, the coefficient matrices of both problems have special structures. Because of this we can develop very simplified versions of the Simplex method for solving these problems that achieve dramatic computational savings

Transportation Problem

Transportation Problem

- A commodity is available at each of the m sources in $a_1, a_2, \dots, a_m$ units
- This commodity is required at each of the n destinations in $b_1, b_2, \dots, b_n$ units
- c_{ij} are the per unit cost of transportation of commodity from the i th source to the j th destination.
- **Problem:** How many units of the given commodity should be transported from each source to each destination so that the total cost of the transportation is minimum and meet the requirements?



Transportation Problem

- Design variables: x_{ij} units of commodity to be transported from the i th source to the j th destination, costing:

$$f = \sum_{i=1}^m \sum_{j=1}^n c_{ij} x_{ij}$$

source *dest*

- Constraints:**
 - From any source, we should not transport more than what is available.
 - To every destination we should transport at least what is required.
 - We cannot transport negative units of commodity.

$$\textcircled{1} \sum_{j=1}^n x_{ij} \leq a_i \quad \textcircled{2} \sum_{i=1}^m x_{ij} \geq b_j \quad \textcircled{3} x_{ij} \geq 0$$

Transportation Problem

- The transportation problem is **balanced** if all source and destination constraints are equalities.

$$\sum_{j=1}^n x_{ij} = a_i \qquad \sum_{i=1}^m x_{ij} = b_j$$

- Such a condition is feasible if and only if:

$$\sum_{i=1}^m a_i = \sum_{j=1}^n b_j$$

- This means the total supply from all sources is equal to the total demand from all destinations

Transportation Problem

$$\sum_{j=1}^n x_{ij} = a_i \qquad \sum_{i=1}^m x_{ij} = b_j$$

$$\begin{array}{rccccccc}
 x_{11} + x_{12} + \dots + x_{1n} & & & & & & = a_1 \\
 & x_{21} + x_{22} + \dots + x_{2n} & & & & & = a_2 \\
 & & & & \dots & & \\
 & & & & & x_{m1} + x_{m2} + \dots + x_{mn} & = a_m \\
 x_{11} & & +x_{21} & & & +x_{m1} & = b_1 \\
 & x_{12} & & +x_{22} & & +x_{m2} & = b_2 \\
 & & & & & & \\
 & & & & & & \\
 & & x_{1n} & & +x_{2n} & & +x_{mn} = b_m.
 \end{array}$$

Transportation Problem

$$A = \begin{bmatrix} 1 & 1 & \dots & 1 & 0 & 0 & \dots & 0 & \dots & 0 & 0 & 0 & \dots & 0 \\ 0 & 0 & \dots & 0 & 1 & 1 & \dots & 1 & \dots & 0 & 0 & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 0 & 0 & 0 & \dots & 0 & \dots & 1 & 1 & \dots & 1 \\ 1 & 0 & \dots & 0 & 1 & 0 & \dots & 0 & \dots & 1 & 0 & \dots & 0 \\ 0 & 1 & \dots & 0 & 0 & 1 & \dots & 0 & \dots & 0 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 & 0 & 0 & \dots & 1 & \dots & 0 & 0 & \dots & 1 \end{bmatrix}$$

- Northwest Corner Rule
- Vogel's Approximation Method



Assignment Problem

Example 1: Job Assignment

- Assigning n persons to n jobs.
- The i th person charges c_{ij} to complete the j th job.

Problem: Determine the assignment of persons to jobs so that the total cost of completing all jobs is minimised

Constraints:

1. Only one person is to be assigned to one job
2. Only one job is to be assigned to a person

	Job A	Job B	Job C	Job D
Worker 1	9	∞	6	4
Worker 2	8	5	∞	7
Worker 3	7	6	5	∞
Worker 4	∞	4	3	2

Example 2: TA Assignment to Courses

	Course X	Course Y	Course Z
TA Alice	3	6	2
TA Bob	5	4	7
TA Carol	6	3	5
TA David	4	2	6

The cost could represent inverse of preference (lower = more preferred). Course requirements:

- X: 1 TA
- Y: 2 TAs
- Z: 1 TA

TA max load: 2 courses each.

Example 3: Hospital-Patient Matching

	Hospital 1	Hospital 2	Hospital 3
Resident A	1	3	2
Resident B	2	1	3
Resident C	3	2	1
Resident D	2	3	1

Cost = rank (lower is better).

Hospital capacities:

- H1: 1
- H2: 2
- H3: 1

Example 4: Flights-to-Gates Assignment

	Gate A	Gate B	Gate C	Gate D
Flight 1	10	8	6	–
Flight 2	9	7	–	5
Flight 3	–	6	4	3
Flight 4	8	–	7	6

Cost = walking distance or turnaround time. "–" means gate is not compatible.

Example 5: Student-Project Allocation

	Project 1	Project 2	Project 3
Student A	1	3	2
Student B	2	1	3
Student C	3	2	1
Student D	1	2	3

Cost = preference rank (lower is better).

Project capacities:

- Project 1: 2
- Project 2: 1
- Project 3: 1

Hungarian Algorithm

- Published in 1955 by Harold Kuhn, who gave it the name “Hungarian method” because the algorithm was largely based on the earlier works of two Hungarian mathematicians, Dénes König and Jenő Egerváry.
- Also called the Munkres assignment algorithm
- $O(n^3)$ vs $O(n!)$



Hungarian Algorithm

Example 1

Hungarian Algorithm

	Flowers	Cake	Photographer
Alice	55	30	25
Bob	40	60	15
Charlie	25	30	45

1. Subtract min of row from each row.
2. Subtract min of col from each col.
3. Draw lines to cover all zeros.
4. If number of lines is equal to the number of rows and cols, then an optimal assignment is an unique zeros assignment to each row and col
Else, subtract the min of all uncovered elements in step 4. Go back to step 3.

Hungarian Algorithm

1. 30 5 0
 25 45 0
 0 5 20

	Flowers	Cake	Photographer
Alice	55	30	25
Bob	40	60	15
Charlie	25	30	45

1. Subtract min of row from each row.
2. Subtract min of col from each col.
3. Draw lines to cover all zeros.
4. If number of lines is equal to the number of rows and cols, then an optimal assignment is an unique zeros assignment to each row and col
Else, subtract the min of all uncovered elements in step 4. Go back to step 3.

Hungarian Algorithm

	Flowers	Cake	Photographer
Alice	55	30	25
Bob	40	60	15
Charlie	25	30	45

1. 30 5 0
 25 45 0
 0 5 20

2. 30 0 0
 25 40 0
 0 0 20

1. Subtract min of row from each row.
2. Subtract min of col from each col.
3. Draw lines to cover all zeros.
4. If number of lines is equal to the number of rows and cols, then an optimal assignment is an unique zeros assignment to each row and col
Else, subtract the min of all uncovered elements in step 4. Go back to step 3.

Hungarian Algorithm

	Flowers	Cake	Photographer
Alice	55	30	25
Bob	40	60	15
Charlie	25	30	45

1. Subtract min of row from each row.
2. Subtract min of col from each col.
3. Draw lines to cover all zeros.
4. If number of lines is equal to the number of rows and cols, then an optimal assignment is an unique zeros assignment to each row and col
Else, subtract the min of all uncovered elements in step 4. Go back to step 3.

1.

30	5	0
25	45	0
0	5	20

2.

30	0	0
25	40	0
0	0	20

3.

30	0	0
25	40	0
0	0	20

Hungarian Algorithm

	Flowers	Cake	Photographer
Alice	55	30	25
Bob	40	60	15
Charlie	25	30	45

1. Subtract min of row from each row.
2. Subtract min of col from each col.
3. Draw lines to cover all zeros.
4. If number of lines is equal to the number of rows and cols, then an optimal assignment is an unique zeros assignment to each row and col
Else, subtract the min of all uncovered elements in step 4. Go back to step 3.

1.

30	5	0
25	45	0
0	5	20

2.

30	0	0
25	40	0
0	0	20

3.

30	0	0
25	40	0
0	0	20

4.

30	0	0
25	40	0
0	0	20

Hungarian Algorithm

	Flowers	Cake	Photographer
Alice	55	30	25
Bob	40	60	15
Charlie	25	30	45

1. Subtract min of row from each row.
2. Subtract min of col from each col.
3. Draw lines to cover all zeros.
4. If number of lines is equal to the number of rows and cols, then an optimal assignment is a unique zeros assignment to each row and col.
Else, subtract the min of all uncovered elements in step 4. Go back to step 3.

1.

30	5	0
25	45	0
0	5	20

2.

30	0	0
25	40	0
0	0	20

3.

30	0	0
25	40	0
0	0	20

4.

30	0	0
25	40	0
0	0	20

Hungarian Algorithm

	Flowers	Cake	Photographer
Alice	55	30	25
Bob	40	60	15
Charlie	25	30	45

1. Subtract min of row from each row.
2. Subtract min of col from each col.
3. Draw lines to cover all zeros.
4. If number of lines is equal to the number of rows and cols, then an optimal assignment is an unique zeros assignment to each row and col
Else, subtract the min of all uncovered elements in step 4. Go back to step 3.

1.

30	5	0
25	45	0
0	5	20

2.

30	0	0
25	40	0
0	0	20

3.

30	0	0
25	40	0
0	0	20

4.

30	0	0
25	40	0
0	0	20

Hungarian Algorithm

	Flowers	Cake	Photographer
Alice	55	30	25
Bob	40	60	15
Charlie	25	30	45

1. Subtract min of row from each row.
2. Subtract min of col from each col.
3. Draw lines to cover all zeros.
4. If number of lines is equal to the number of rows and cols, then an optimal assignment is an unique zeros assignment to each row and col
Else, subtract the min of all uncovered elements in step 4. Go back to step 3.

1.

30	5	0
25	45	0
0	5	20

2.

30	0	0
25	40	0
0	0	20

3.

30	0	0
25	40	0
0	0	20

4.

30	0	0
25	40	0
0	0	20

Hungarian Algorithm

	Flowers	Cake	Photographer
Alice	55	30	25
Bob	40	60	15
Charlie	25	30	45

1. Subtract min of row from each row.
2. Subtract min of col from each col.
3. Draw lines to cover all zeros.
4. If number of lines is equal to the number of rows and cols, then an optimal assignment is an unique zeros assignment to each row and col
Else, subtract the min of all uncovered elements in step 4. Go back to step 3.

1. $\begin{matrix} 30 & 5 & 0 \\ 25 & 45 & 0 \\ 0 & 5 & 20 \end{matrix}$

2. $\begin{matrix} 30 & 0 & 0 \\ 25 & 40 & 0 \\ 0 & 0 & 20 \end{matrix}$

3. $\begin{array}{ccc|c} 30 & 0 & 0 & \\ 25 & 40 & 0 & \\ 0 & 0 & 20 & \end{array}$

4. $\begin{matrix} 30 & 0 & 0 \\ 25 & 40 & 0 \\ 0 & 0 & 20 \end{matrix}$

Hungarian Algorithm

Example 2

Hungarian Algorithm

	Flowers	Cake	Photographer
Alice	30	25	10
Bob	15	10	20
Charlie	25	20	15

1. Subtract min of row from each row.
2. Subtract min of col from each col.
3. Draw lines to cover all zeros.
4. If number of lines is equal to the number of rows and cols, then an optimal assignment is an unique zeros assignment to each row and col
Else, subtract the min of all uncovered elements in step 4. Go back to step 3.

Hungarian Algorithm

1. 20 15 0
 5 0 10
 10 5 0

	Flowers	Cake	Photographer
Alice	30	25	10
Bob	15	10	20
Charlie	25	20	15

1. Subtract min of row from each row.
2. Subtract min of col from each col.
3. Draw lines to cover all zeros.
4. If number of lines is equal to the number of rows and cols, then an optimal assignment is an unique zeros assignment to each row and col
Else, subtract the min of all uncovered elements in step 4. Go back to step 3.

Hungarian Algorithm

	Flowers	Cake	Photographer
Alice	30	25	10
Bob	15	10	20
Charlie	25	20	15

1. 20 15 0
 5 0 10
 10 5 0

2. 15 15 0
 0 0 10
 5 5 0

1. Subtract min of row from each row.
2. Subtract min of col from each col.
3. Draw lines to cover all zeros.
4. If number of lines is equal to the number of rows and cols, then an optimal assignment is an unique zeros assignment to each row and col
Else, subtract the min of all uncovered elements in step 4. Go back to step 3.

Hungarian Algorithm

	Flowers	Cake	Photographer
Alice	30	25	10
Bob	15	10	20
Charlie	25	20	15

1. Subtract min of row from each row.
2. Subtract min of col from each col.
3. Draw lines to cover all zeros.
4. If number of lines is equal to the number of rows and cols, then an optimal assignment is an unique zeros assignment to each row and col
Else, subtract the min of all uncovered elements in step 4. Go back to step 3.

1. 20 15 0
 5 0 10
 10 5 0

2. 15 15 0
 0 0 10
 5 5 0

3. 15 15 0
— 0 — 0 — 10 —
 5 5 0

Hungarian Algorithm

	Flowers	Cake	Photographer
Alice	30	25	10
Bob	15	10	20
Charlie	25	20	15

1. Subtract min of row from each row.
2. Subtract min of col from each col.
3. Draw lines to cover all zeros.
4. If number of lines is equal to the number of rows and cols, then an optimal assignment is an unique zeros assignment to each row and col
Else, subtract the min of all uncovered elements in step 4. Go back to step 3.

1. 20 15 0
 5 0 10
 10 5 0

2. 15 15 0
 0 0 10
 5 5 0

3. 15 15 0
— 0 — 0 — 10 —
 5 5 0

4. 10 10 0
 0 0 5
 0 0 0

Hungarian Algorithm

	Flowers	Cake	Photographer
Alice	30	25	10
Bob	15	10	20
Charlie	25	20	15

1. Subtract min of row from each row.
2. Subtract min of col from each col.
3. Draw lines to cover all zeros.
4. If number of lines is equal to the number of rows and cols, then an optimal assignment is an unique zeros assignment to each row and col
Else, subtract the min of all uncovered elements in step 4. Go back to step 3.

1. 20 15 0
 5 0 10
 10 5 0

2. 15 15 0
 0 0 10
 5 5 0

3. 15 15 0
— 0 0 10 —
 5 5 0

4. — 10 10 0 —
 — 0 0 5 —
 — 0 0 0 —

Hungarian Algorithm

	Flowers	Cake	Photographer
Alice	30	25	10
Bob	15	10	20
Charlie	25	20	15

1. Subtract min of row from each row.
2. Subtract min of col from each col.
3. Draw lines to cover all zeros.
4. If number of lines is equal to the number of rows and cols, then an optimal assignment is an unique zeros assignment to each row and col
Else, subtract the min of all uncovered elements in step 4. Go back to step 3.

1. 20 15 0
 5 0 10
 10 5 0

2. 15 15 0
 0 0 10
 5 5 0

3. 15 15 0
— 0 — 0 — 10 —
 5 5 0

4. 10 10 0
— 0 — 0 — 5 —
— 0 — 0 — 0 —

5. 10 10 0
 0 0 5
 0 0 0

Hungarian Algorithm

	Flowers	Cake	Photographer
Alice	30	25	10
Bob	15	10	20
Charlie	25	20	15

1. Subtract min of row from each row.
2. Subtract min of col from each col.
3. Draw lines to cover all zeros.
4. If number of lines is equal to the number of rows and cols, then an optimal assignment is an unique zeros assignment to each row and col
Else, subtract the min of all uncovered elements in step 4. Go back to step 3.

1.

20	15	0
5	0	10
10	5	0

2.

15	15	0
0	0	10
5	5	0

3.

15	15	0
0	0	10
5	5	0

4.

10	10	0
0	0	5
0	0	0

5.

10	10	0
0	0	5
0	0	0

Hungarian Algorithm

	Flowers	Cake	Photographer
Alice	30	25	10
Bob	15	10	20
Charlie	25	20	15

1. Subtract min of row from each row.
2. Subtract min of col from each col.
3. Draw lines to cover all zeros.
4. If number of lines is equal to the number of rows and cols, then an optimal assignment is an unique zeros assignment to each row and col
Else, subtract the min of all uncovered elements in step 4. Go back to step 3.

1. 20 15 0
 5 0 10
 10 5 0

2. 15 15 0
 0 0 10
 5 5 0

3. 15 15 0
— 0 — 0 — 10 —
 5 5 0

4. 10 10 0
— 0 — 0 — 5 —
— 0 — 0 — 0 —

5. 10 10 0
 0 0 5
0 0 0

Hungarian Algorithm

	Flowers	Cake	Photographer
Alice	30	25	10
Bob	15	10	20
Charlie	25	20	15

1. Subtract min of row from each row.
2. Subtract min of col from each col.
3. Draw lines to cover all zeros.
4. If number of lines is equal to the number of rows and cols, then an optimal assignment is an unique zeros assignment to each row and col
Else, subtract the min of all uncovered elements in step 4. Go back to step 3.

1.

20	15	0
5	0	10
10	5	0

2.

15	15	0
0	0	10
5	5	0

3.

15	15	0
0	0	10
5	5	0

4.

10	10	0
0	0	5
0	0	0

5.

10	10	0
0	0	5
0	0	0

Hungarian Algorithm

	Flowers	Cake	Photographer
Alice	30	25	10
Bob	15	10	20
Charlie	25	20	15

1. Subtract min of row from each row.
2. Subtract min of col from each col.
3. Draw lines to cover all zeros.
4. If number of lines is equal to the number of rows and cols, then an optimal assignment is an unique zeros assignment to each row and col
Else, subtract the min of all uncovered elements in step 4. Go back to step 3.

1.

20	15	0
5	0	10
10	5	0

2.

15	15	0
0	0	10
5	5	0

3.

15	15	0
0	0	10
5	5	0

4.

10	10	0
0	0	5
0	0	0

5.

10	10	0
0	0	5
0	0	0

Num Worker $\neq$ Num Tasks

https://en.wikipedia.org/wiki/Assignment_problem#/media/File:Hungarian_algorithm_unbalanced_assignment_problem_example.svg

1 Given matrix of costs

Worker	Task		
	I	II	III
Ali	8	4	7
Baba	5	2	3
Curi	9	6	7
Durian	9	4	8

Make square with dummy column.
Subtract minimum for each column:

5	2	3	
3	2	4	0
0	0	0	0
4	4	4	0
4	2	5	0

2

For each row with exactly one uncanceled 0, cancel its column

3

3	2	4	0
0	0	0	0
4	4	4	0
4	2	5	0

For each column with exactly one uncanceled 0, cancel its row

4

3	2	4	0
0	0	0	0
4	4	4	0
4	2	5	0

2 (cancels) < 4 (rows), hence subtract smallest uncanceled (2) from uncanceled; add to intersects

5

1	0	2	0
0	0	0	2
2	2	2	0
2	0	3	0

Remove cancellations.
For each row with exactly one uncanceled 0, cancel its column

6

1	0	2	0
0	0	0	2
2	2	2	0
2	0	3	0

7

1	0	2	0
0	0	0	2
2	2	2	0
2	0	3	0

For each column with exactly one uncanceled 0, cancel its row

8

1	0	2	0
0	0	0	2
2	2	2	0
2	0	3	0

3 (cancels) < 4 (rows), hence subtract smallest uncanceled (1) from uncanceled; add to intersects

9

0	0	1	0
0	1	0	3
1	2	1	0
1	0	2	0

Remove cancellations.
For each row with exactly one uncanceled 0, cancel its column

10

0	0	1	0
0	1	0	3
1	2	1	0
1	0	2	0

11

0	0	1	0
0	1	0	3
1	2	1	0
1	0	2	0

For each column with exactly one uncanceled 0, cancel its row

12

0	0	1	0
0	1	0	3
1	2	1	0
1	0	2	0

As a 0 remains uncanceled, for each row with exactly one uncanceled 0, cancel its column

13

0	0	1	0
0	1	0	3
1	2	1	0
1	0	2	0

4 (cancels) = 4 (rows), hence assignment are marked cells

14

Worker	Task		
	I	II	III
Ali	8	4	7
Baba	5	2	3
Curi	9	6	7
Durian	9	4	8

Hungarian Algorithm

https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.linear_sum_assignment.html



1.13.0 (stable) ▾

Installing User Guide API reference Building from source Development Release notes

scipy
scipy.cluster
scipy.constants
scipy.datasets
scipy.fft
scipy.fftpack
scipy.integrate
scipy.interpolate
scipy.io
scipy.linalg
scipy.misc
scipy.ndimage
scipy.odr
scipy.optimize
scipy.signal
scipy.sparse
scipy.spatial
scipy.special
scipy.stats

🏠 > SciPy API > Optimization and root finding (`scipy.optimize`) > `scipy.optimi...`

scipy.optimize.linear_sum_assignment

`scipy.optimize.linear_sum_assignment()`

Solve the linear sum assignment problem.

Parameters:

cost_matrix : *array*

The cost matrix of the bipartite graph.

maximize : *bool (default: False)*

Calculates a maximum weight matching if true.

Returns:

row_ind, col_ind : *array*

An array of row indices and one of corresponding column indices giving the optimal assignment. The cost of the assignment can be computed as

`cost_matrix[row_ind, col_ind].sum()`. The row indices will be sorted; in the

case of a square cost matrix they will be equal to

`numpy.arange(cost_matrix.shape[0])`.

Extensions and More to Explore

- Very close connections between the transportation and assignment problems to graph theory.
- A very natural extension of the transportation problem is the multi-commodity transportation problem.
- If we allow that an item may reach a destination via another source or via another destination or a combination of these, then the transportation problem is called the **transshipment problem**. The transshipment problem can be transformed into the usual transportation problem with $m+n$ sources and $m+n$ destination and then solved by the usual algorithm.
- The assignment problem is also related to the **travelling salesperson problem** (TSP). Solving an n -city TSP is equivalent to finding a cyclic solution to the usual n -person, n -jobs assignment problem.

The Job Assignment Example LP Formulation

- Assigning n persons to n jobs.
- The i th person charges c_{ij} to complete the j th job.

Problem: Determine the assignment of persons to jobs so that the total cost of completing all jobs is minimised

Constraints:

1. Only one person is to be assigned to one job
2. Only one job is to be assigned to a person

	Job A	Job B	Job C	Job D
Worker 1	9	∞	6	4
Worker 2	8	5	∞	7
Worker 3	7	6	5	∞
Worker 4	∞	4	3	2

The Job Assignment Example LP Formulation

Let $x_{ij} = \begin{cases} 1 & \text{if worker } i \text{ is assigned to job } j \\ 0 & \text{otherwise} \end{cases}$

Variables:

$x_{ij} \in \{0, 1\}$ for all feasible (i, j)

Objective Function:

Minimize $Z = 9x_{11} + 6x_{13} + 4x_{14} + 8x_{21} + 5x_{22} + 7x_{24} + 7x_{31} + 6x_{32} + 5x_{33} + 4x_{42} + 3x_{43} + 2x_{44}$

Constraints:

- Each worker is assigned to exactly one job:

$$x_{11} + x_{13} + x_{14} = 1 \quad (\text{Worker 1})$$

$$x_{21} + x_{22} + x_{24} = 1 \quad (\text{Worker 2})$$

$$x_{31} + x_{32} + x_{33} = 1 \quad (\text{Worker 3})$$

$$x_{42} + x_{43} + x_{44} = 1 \quad (\text{Worker 4})$$

- Each job is assigned to exactly one worker:

$$x_{11} + x_{21} + x_{31} = 1 \quad (\text{Job A})$$

$$x_{22} + x_{32} + x_{42} = 1 \quad (\text{Job B})$$

$$x_{13} + x_{33} + x_{43} = 1 \quad (\text{Job C})$$

$$x_{14} + x_{24} + x_{44} = 1 \quad (\text{Job D})$$

Job A	Job B	Job C	Job D
9	∞	6	4
8	5	∞	7
7	6	5	∞
∞	4	3	2

Assignment Problem

- Design variables: x_{ij}

$$x_{ij} = \begin{cases} 1 & \text{if the } i^{\text{th}} \text{ person is assigned the } j^{\text{th}} \text{ job} \\ 0 & \text{otherwise.} \end{cases}$$

$$\text{Min} \quad z = \sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij}$$

subject to

$$\sum_{i=1}^n x_{ij} = 1 \quad (j = 1, 2, \dots, n)$$

$$\sum_{j=1}^n x_{ij} = 1 \quad (i = 1, 2, \dots, n)$$

$$x_{ij} = 0 \text{ or } 1 \quad \text{for all } i \text{ and } j.$$

Discrete Optimisation

Outline

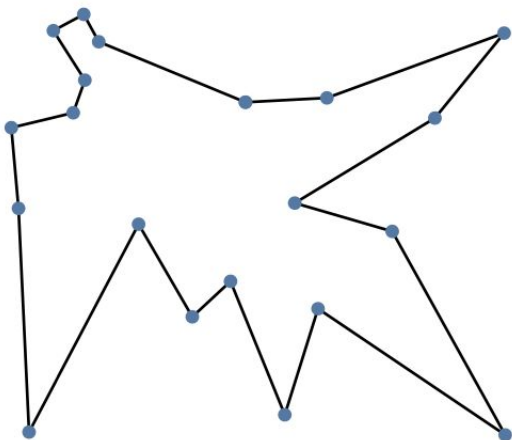
- Discrete Optimisation Motivation
- Branch and Bound
- Greedy Algorithm

Discrete Optimisation

- Common examples of discrete optimisation include:
 - Scheduling
 - Network problems
 - Resource allocation
- Concrete examples include:
 - Power plants on/off
 - Number of wheels on a vehicle
 - The material in a structure that is restricted to titanium, steel, or aluminum
 - The traveling salesperson problem (TSP)
- Discrete optimization is also known as:
integer programming or **combinatorial optimisation**
- Problems with both continuous and discrete variables are referred to as **mixed-integer programming**

Discrete Optimisation Complexity

- Discrete optimisation is nondeterministic polynomial-time complete, or **NP-complete**: we can easily verify a solution, but there is no known approach to find a solution efficiently.
- The time required to solve discrete problems become much worse as the problem size grows. So exhaustive search is well, exhausting!
- Example: TSP with 20 cities: $(20-1)! = 19!$ possibilities



```
[In [1]: import numpy as np
```

```
[In [2]: np.math.factorial(19)
```

```
Out[2]: 121645100408832000
```

```
[In [3]: np.math.factorial(19) / 1e9 / (60*60*24*365)
```

```
Out[3]: 3.8573408298082192
```

```
[In [4]: np.math.factorial(19) / 3e9 / (60*60*24)/28
```

```
Out[4]: 16.76106432
```

Avoiding Discrete Optimisation

- Even though a discrete optimisation problem limits the options and thus conceptually sounds easier to solve, discrete optimisation problems are usually much more challenging to solve than continuous problems.
- Methods to avoid discrete optimisation include:
 - **Exhaustive search:** We may have only a few discrete variables with few options. In that case, enumerating all options is possible.
 - **Rounding:** We can optimise the discrete design variables as if they were continuous and round the optimal design variable to integer values afterward.
 - **Dynamic rounding:** Round only a subset of the discrete variables, fix them, and re-optimizes the remaining variables using continuous optimisation. The process is repeated until all discrete variables are fixed.

Embracing Discrete Optimisation

- We can convert a discrete optimisation problem into a **binary discrete** one by adding additional variables and constraints. Although the new problem is larger, it is usually far easier to solve.
- Example: An engineering structure may use one of n different materials. Convert the problem to have n binary variables x_i
- We would also need to add an additional constraint to make sure that one (and only one) material is selected:

$$\sum_{i=1}^n x_i = 1$$

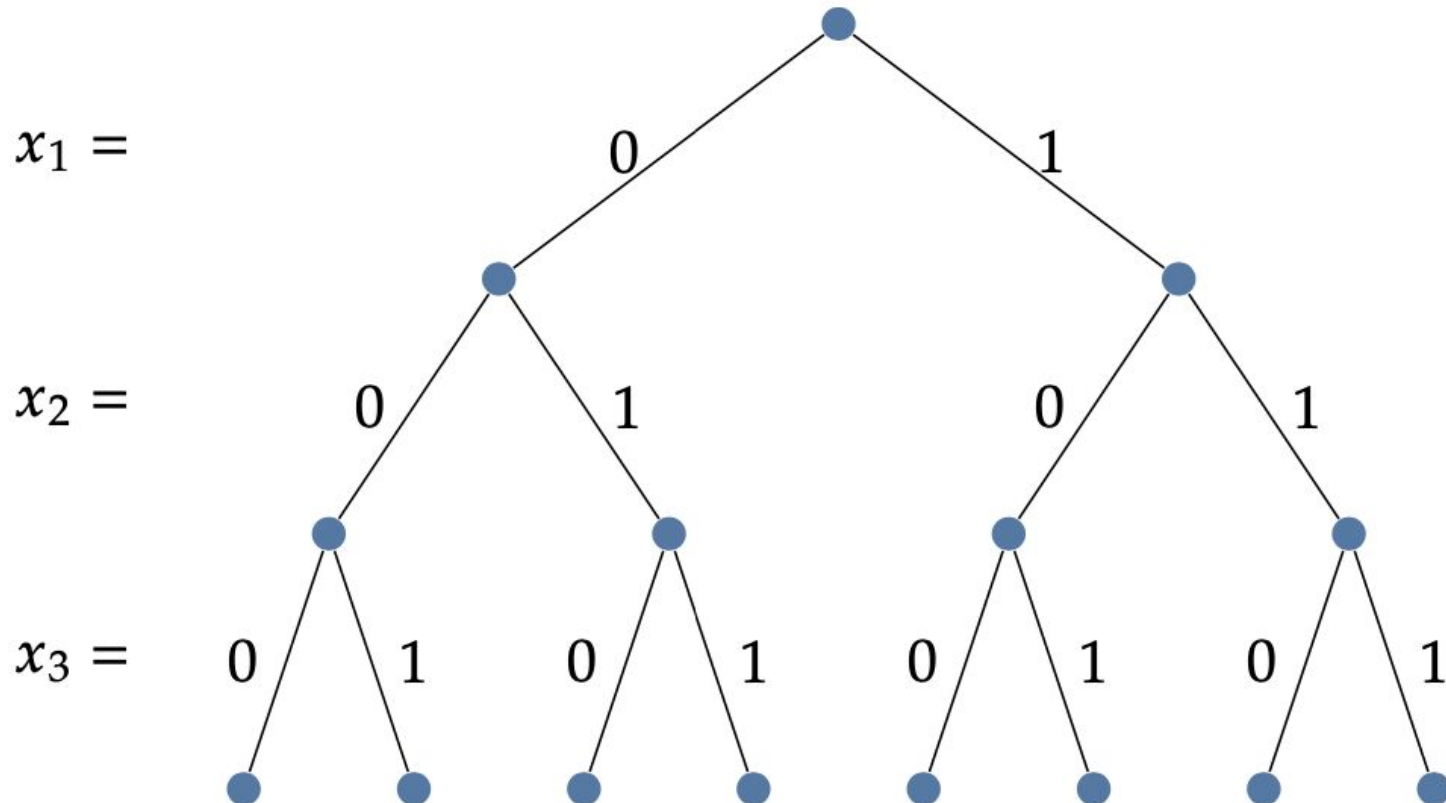
Branch and Bound (B&B)

Branch and Bound

- B&B was first proposed by Ailsa Land and Alison Doig at the London School of Economics in 1960 for discrete programming and has become the most commonly used tool for solving NP-hard optimisation problems.
- B&B algorithm involves:
 - **Branching:** Splits the search space into smaller spaces, then minimise $f(x)$ in these smaller spaces.
 - **Bounding:** Keeps track of the minimum so far and use this to eliminate branches
 - **Relaxation:** Aims to construct an approximation of the original optimisation problem that is easier to solve.

Branching

- Splits the search space into smaller spaces, then minimise $f(x)$ in these smaller spaces.
- Branching is done by adding constraints



Bounding

- Branching by itself would amount to brute-force enumeration of candidate solutions and testing them all.
- To improve on the performance of an exhaustive search, the B&B algorithm keeps track of bounds on the minimum, and uses this bound to **prune** the search space, eliminating candidate solutions that we know will not contain an optimal solution.

Relaxation

- Approximate the original optimisation problem such that is easier to solve.
- Such approximations are often accomplished by removing, or "relaxing" constraints.
- For example if $x_i = 0$ or 1 relax this to

$$0 \leq x_i \leq 1$$

Example 1

$$\begin{array}{ll}\text{minimize} & -2.5x_1 - 1.1x_2 - 0.9x_3 - 1.5x_4 \\ \text{subject to} & 4.3x_1 + 3.8x_2 + 1.6x_3 + 2.1x_4 \leq 9.2 \\ & 4x_1 + 2x_2 + 1.9x_3 + 3x_4 \leq 9 \\ & x_i \in \{0, 1\} \text{ for all } i.\end{array}$$

- Relax the binary problem and solve with continuous bounds:
 $0 \leq x_i \leq 1$

$$x^* = [1, 0.5274, 0.4975, 1]$$

$$f^* = -5.0279.$$

Example 1

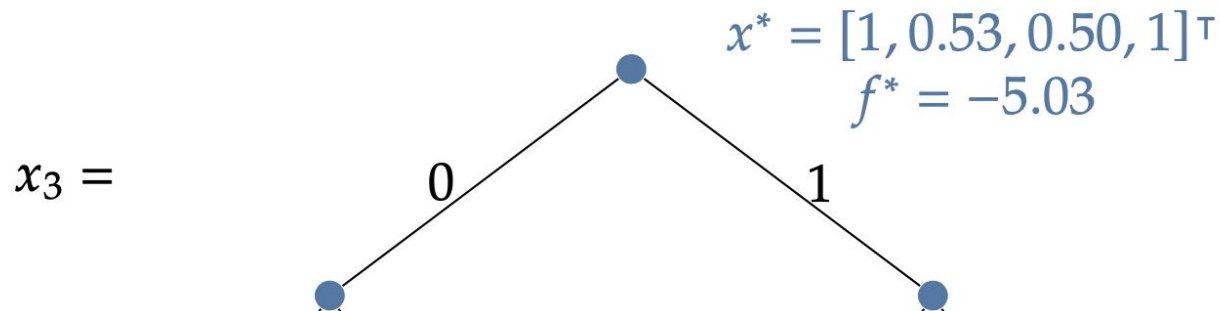
- $$\begin{aligned} x^* &= [1, 0.53, 0.50, 1]^\top \\ f^* &= -5.03 \end{aligned}$$

Example 1

- $$\begin{aligned} x^* &= [1, 0.53, 0.50, 1]^\top \\ f^* &= -5.03 \end{aligned}$$

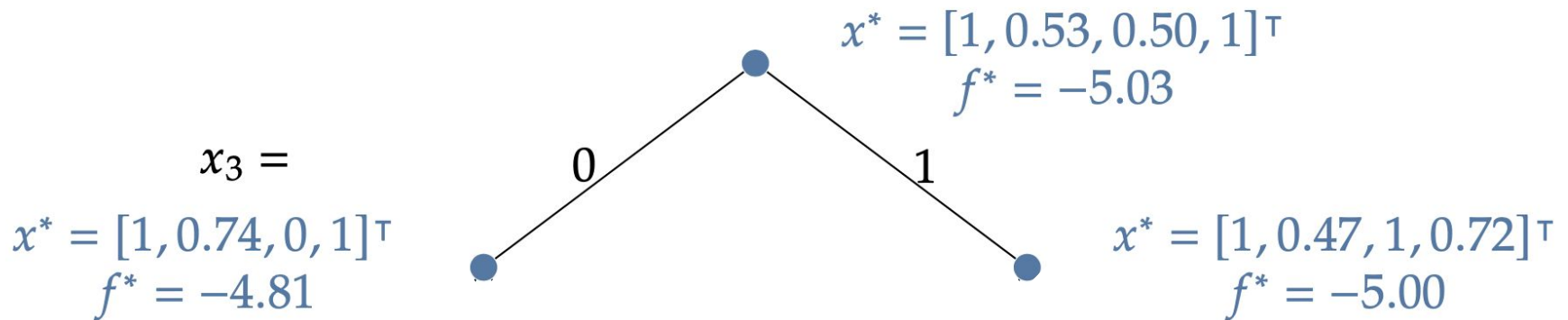
- Branch on the variable with the most fractional component

Example 1



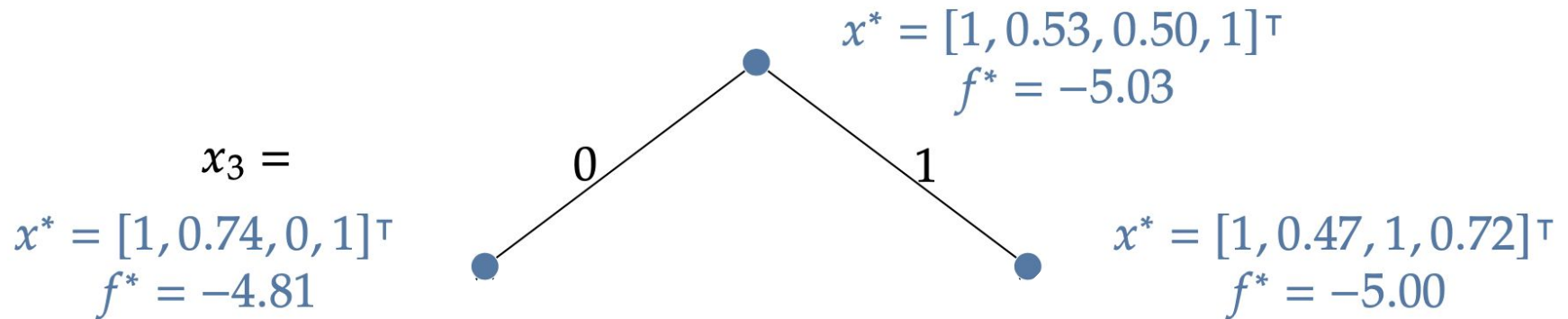
- Branch on the variable with the most fractional component.
- Add the constraints $x_3 = 0$ and $x_3 = 1$ to form two branches.

Example 1



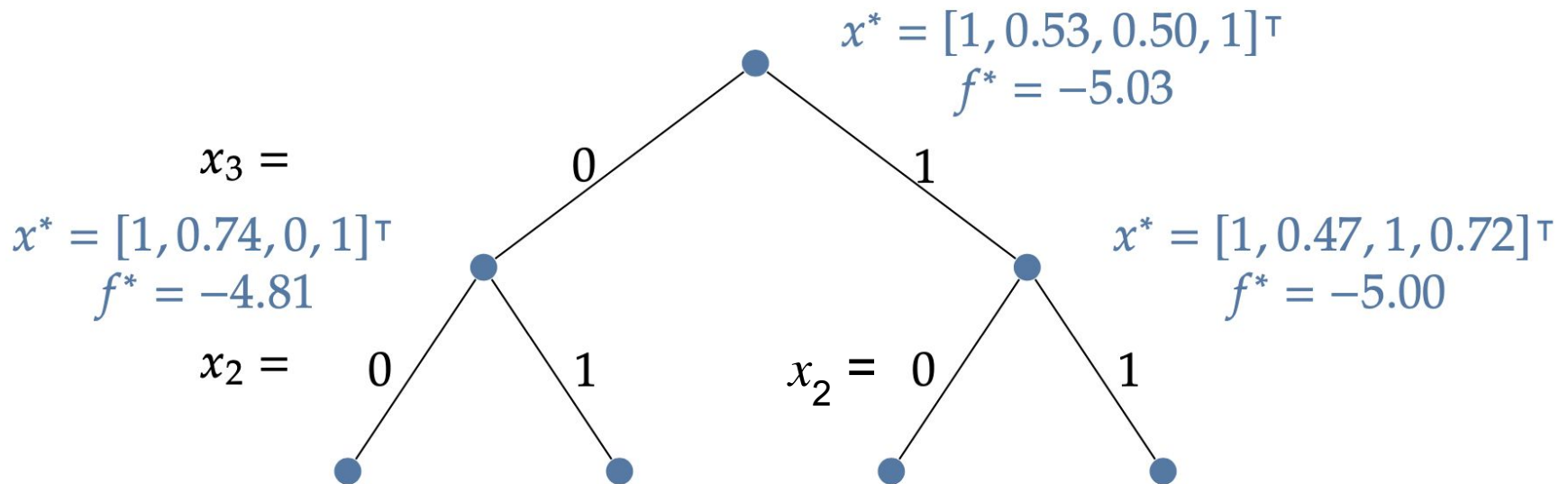
- Branch on the variable with the most fractional component.
- Add the constraints $x_3 = 0$ and $x_3 = 1$ to form two branches.
- Solve

Example 1



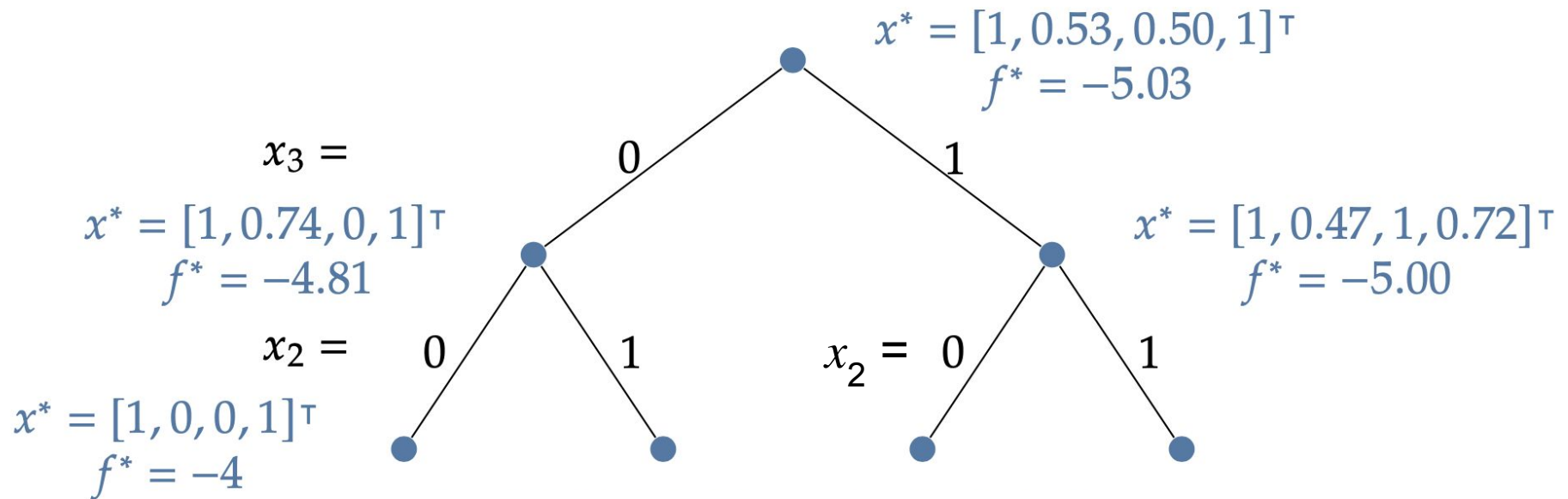
- Neither of these optimizations yields all binary values, so we have to branch both of them.

Example 1



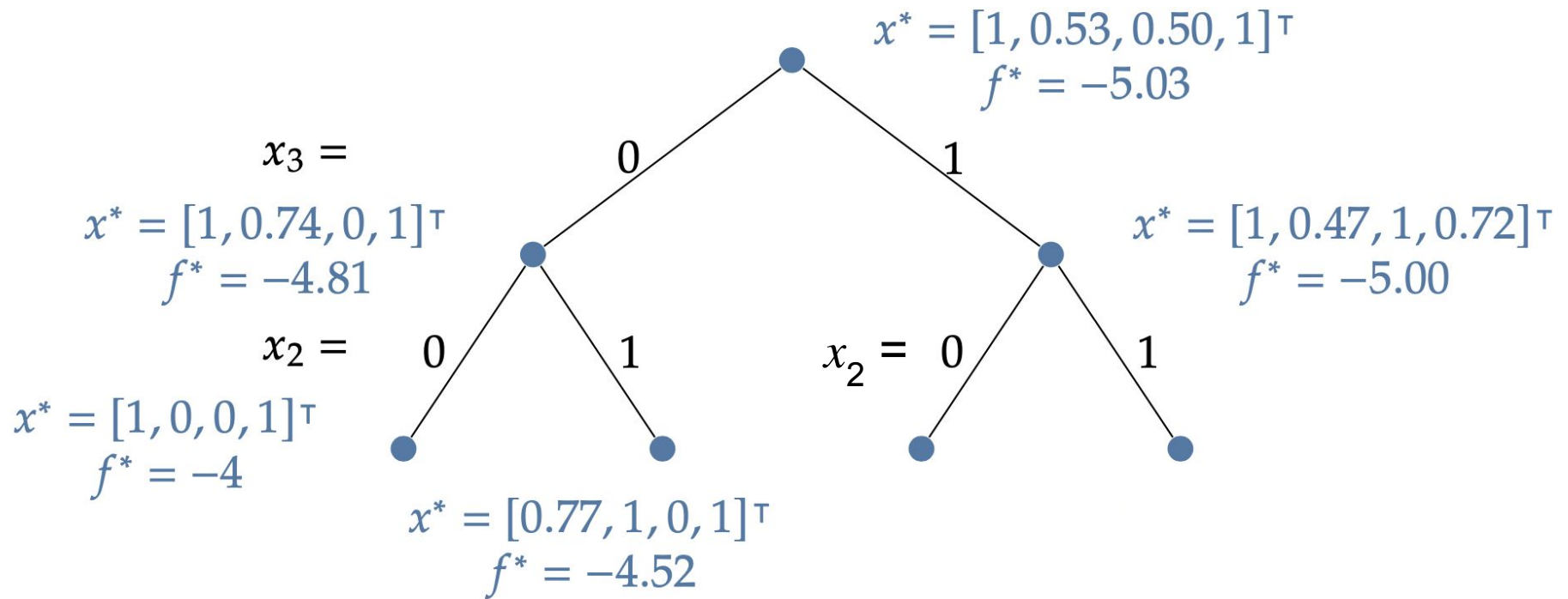
- Neither of these optimizations yields all binary values, so we have to branch both of them.
- In this case, the left node branches on x_2 (the only fractional component), and the right node also branches on x_2 (the most fractional component).

Example 1



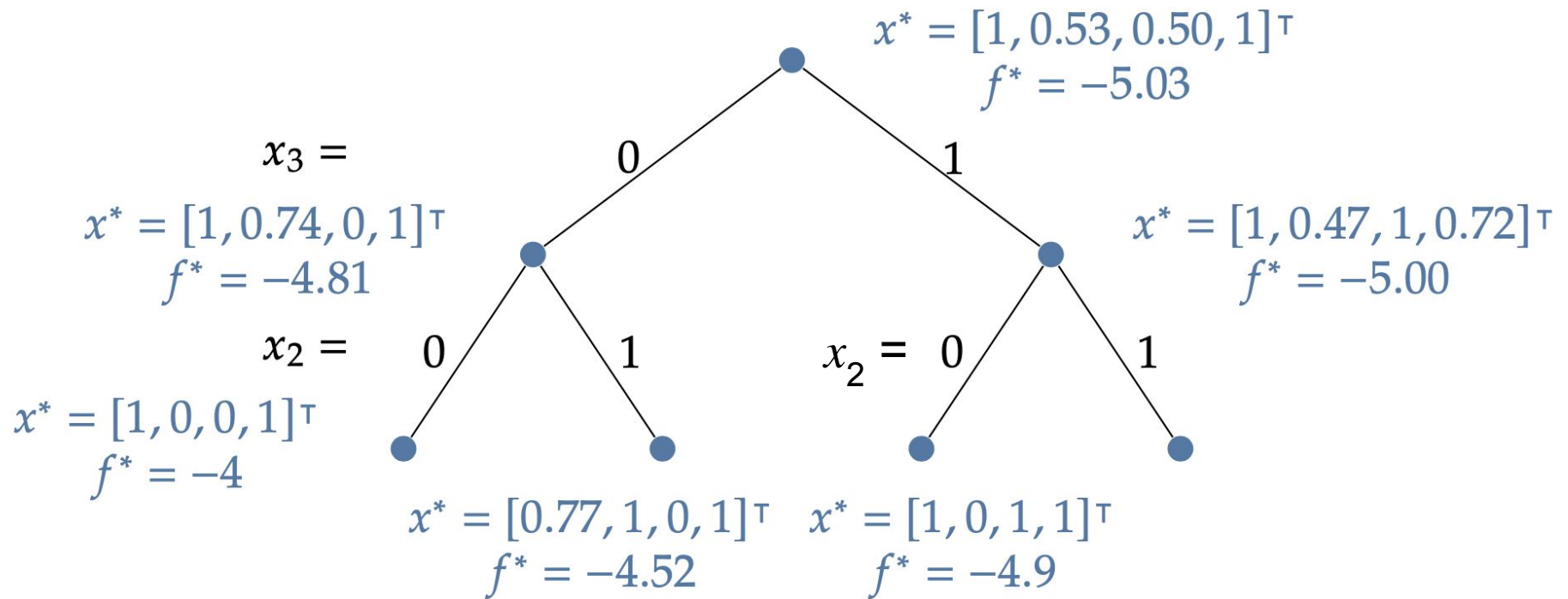
- The first branch yields a feasible binary solution. The corresponding objective function value $f = -4$ is saved as the best value so far. There is no need to continue on this branch because the solution cannot be improved on this particular branch.

Example 1



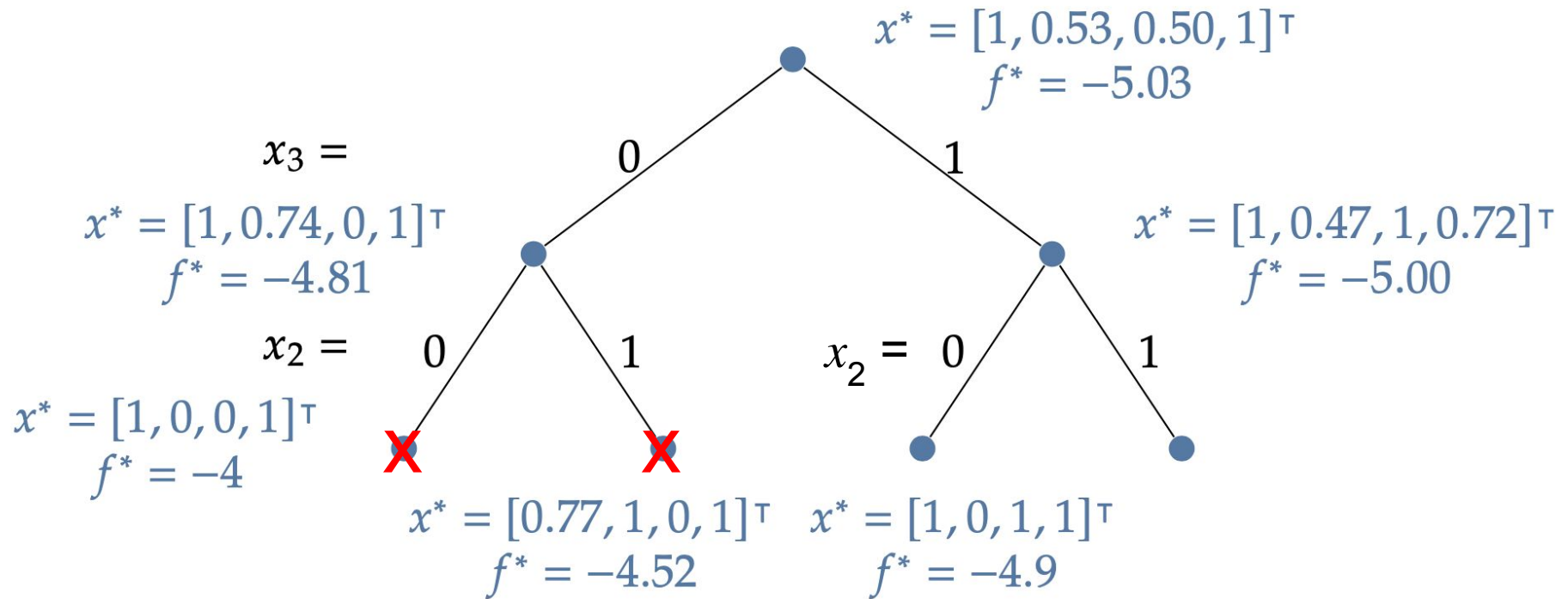
- Solve branch 2

Example 1



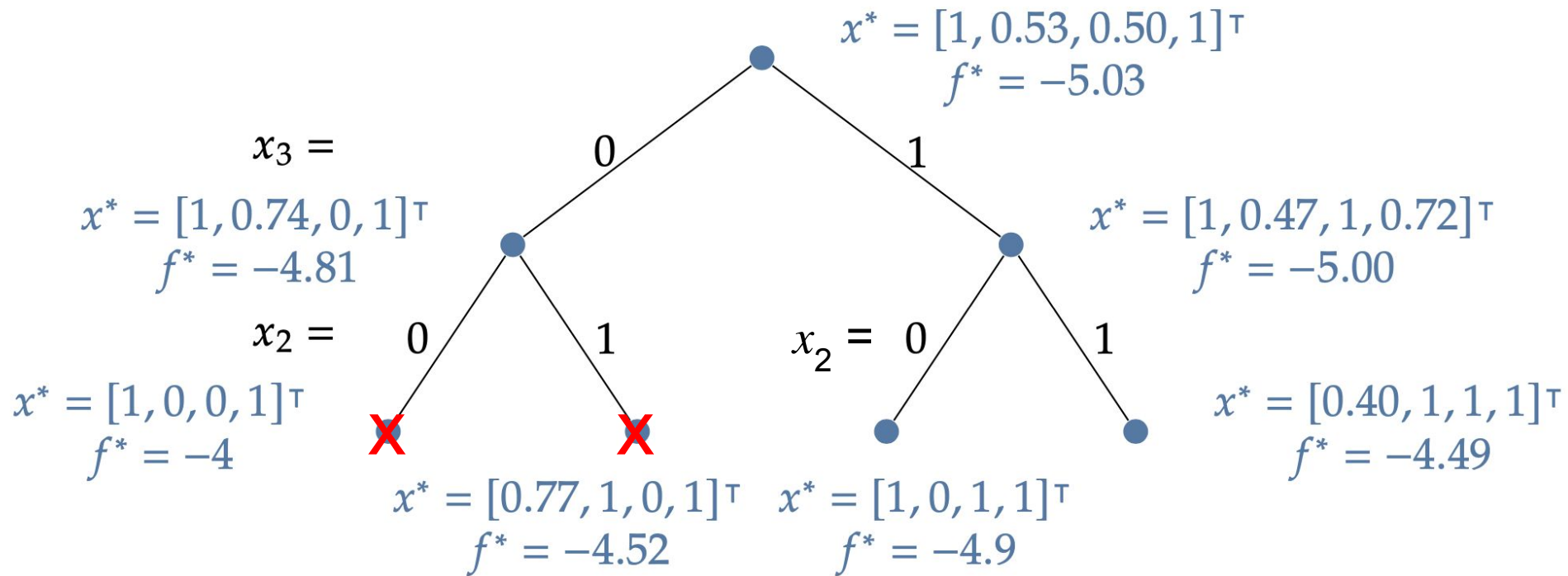
- Solve branch 3 - yields a feasible binary solution.

Example 1



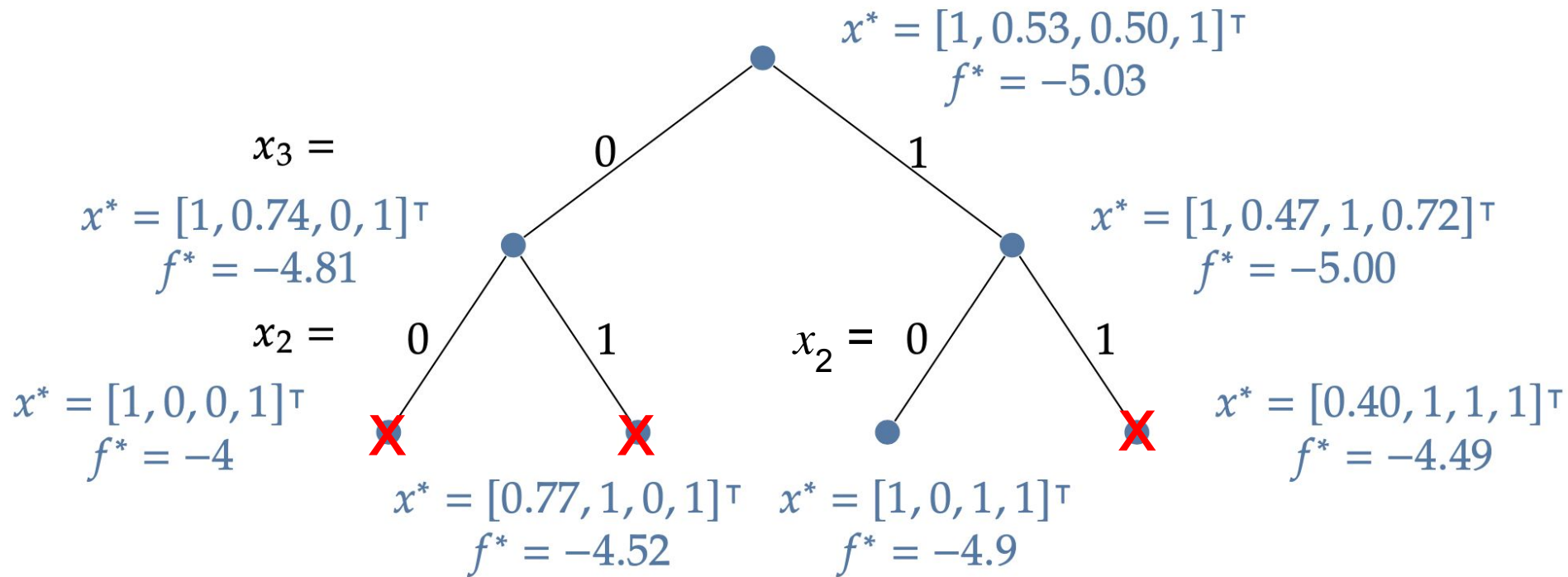
- Branch 1 and 2 have a lower-bound that is worse than the best solution so far. Thus, we can prune both of these branches.

Example 1



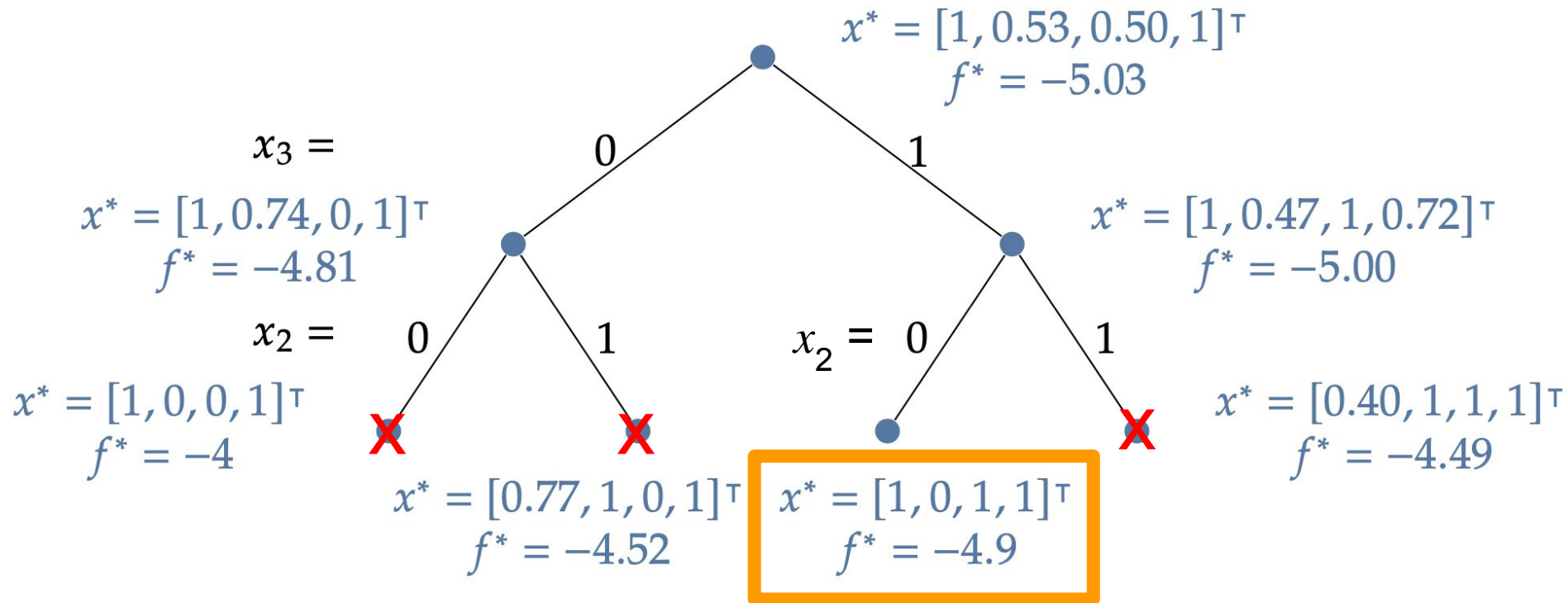
- Solve branch 4

Example 1



- Branch 4 has a lower bound that is worse than the best solution so far. Thus, we can prune that branch too.

Example 1



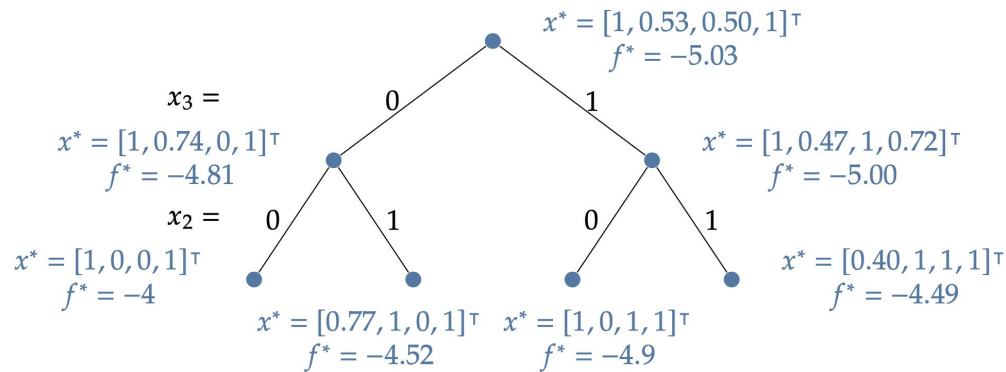
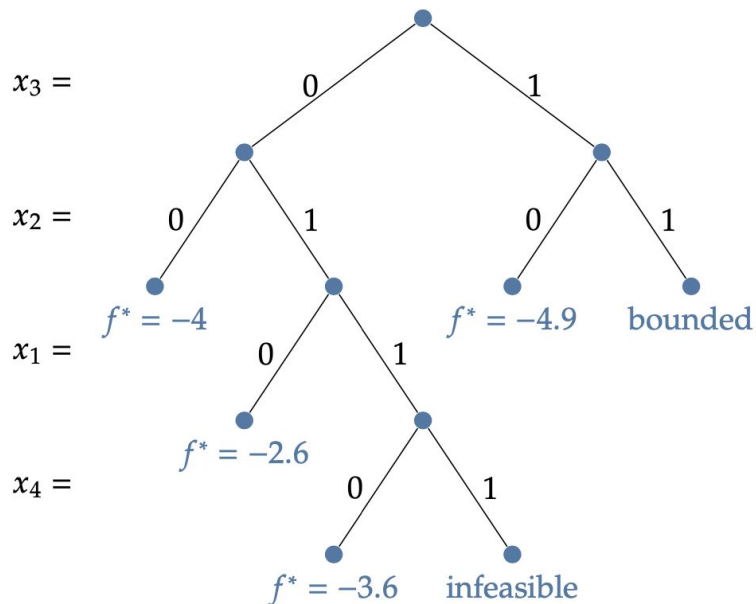
- All branches have been pruned, we have solved the original problem.

What just happened

- The solution from a relaxed problem provides a lower bound
- the best that could be achieved if continued on that branch.
- This is because adding constraints (i.e. branching) always leads to a solution that is either the same or worse, never better.
- One common strategy is to branch on the variable with the largest fractional component.
- For example, if $x = [1.0, 0.4, 0.9, 0.0]$, we could branch on x_2 or x_3 because both are fractional. But we are more likely to force the algorithm to make faster progress by branching on variables that are furthest away from integers. In this case, that value would be $x_2 = 0.4$.

Depth-first vs Breadth-first

- **Depth-first** strategy continues as far down as possible (e.g., by always branching left) until it cannot go further, and then it follows right branches.
- **Breadth-first** strategy explores all nodes on a given level before increasing depth.



Depth-first vs Breadth-first

- Depth-first is a common strategy because, in the absence of more information about a problem, it is most likely to be the fastest way to find a solution - reaching the bottom of the tree generally forces a solution.
- Finding a solution quickly is desirable because this solution can then be used as a lower bound on other branches - so they can be pruned.
- The depth-first strategy requires less memory storage because breadth-first must maintain a longer history as the number of branches increases. In contrast, depth-first only requires node storage equal to the number of branch levels.

Mixed-Integer Programming

- We can use the same B&B procedure with integer variables.
- Instead of branching with two equality constraints ($x_i = 0$ and $x_i = 1$), we **branch with two inequality constraints**.
- Example, if the variable we branched on is $x_1 = 3.4$, we would branch with two new problems with the following constraints: $x_1 \leq 3$ and $x_1 \geq 4$.

Example 2

$$\begin{array}{ll}\text{minimize} & -x_1 - 2x_2 - 3x_3 - 1.5x_4 \\ \text{subject to} & x_1 + x_2 + 2x_3 + 2x_4 \leq 10 \\ & 7x_1 + 8x_2 + 5x_3 + x_4 = 31.5 \\ & x_i \in \mathbb{Z}^+ \text{ for } i = 1, 2, 3 \\ & x_4 \geq 0.\end{array}$$

- Solve the relaxed problem, replacing the integer constraints with a continuous constraint: $x_i \geq 0$

$$x^* = [0, 1.1818, 4.4091, 0], \quad f^* = -15.59$$

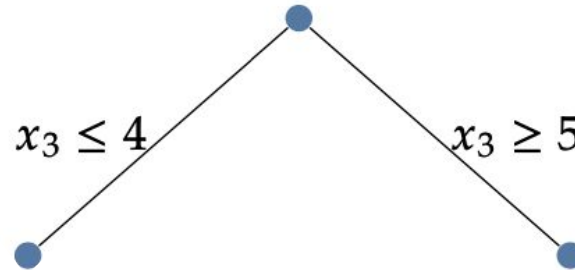
Example 2

$$x^* = [0, 1.1818, 4.4091, 0], \quad f^* = -15.59$$



Example 2

$$x^* = [0, 1.1818, 4.4091, 0], \quad f^* = -15.59$$

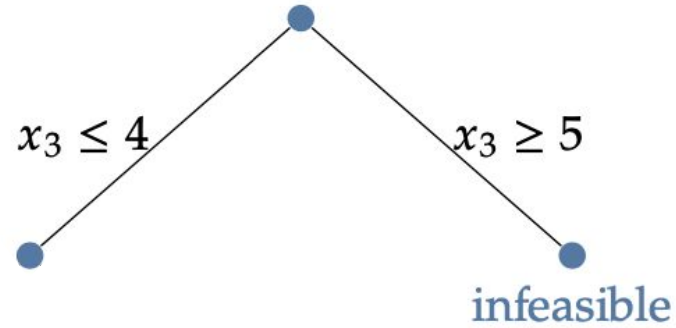


Example 2

$$x^* = [0, 1.1818, 4.4091, 0], \quad f^* = -15.59$$

$$x^* = [0, 1.4, 4, 0.3]$$

$$f^* = -15.25$$

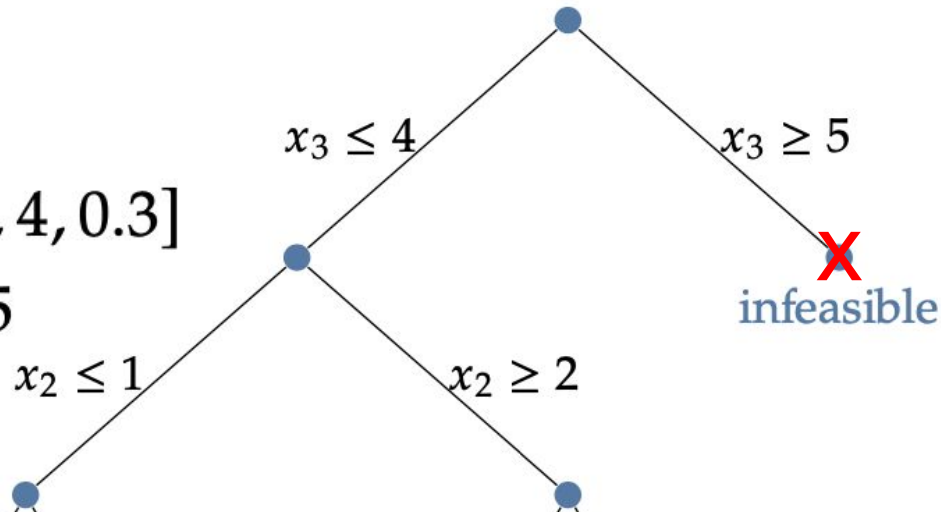


Example 2

$$x^* = [0, 1.1818, 4.4091, 0], \quad f^* = -15.59$$

$$x^* = [0, 1.4, 4, 0.3]$$

$$f^* = -15.25$$

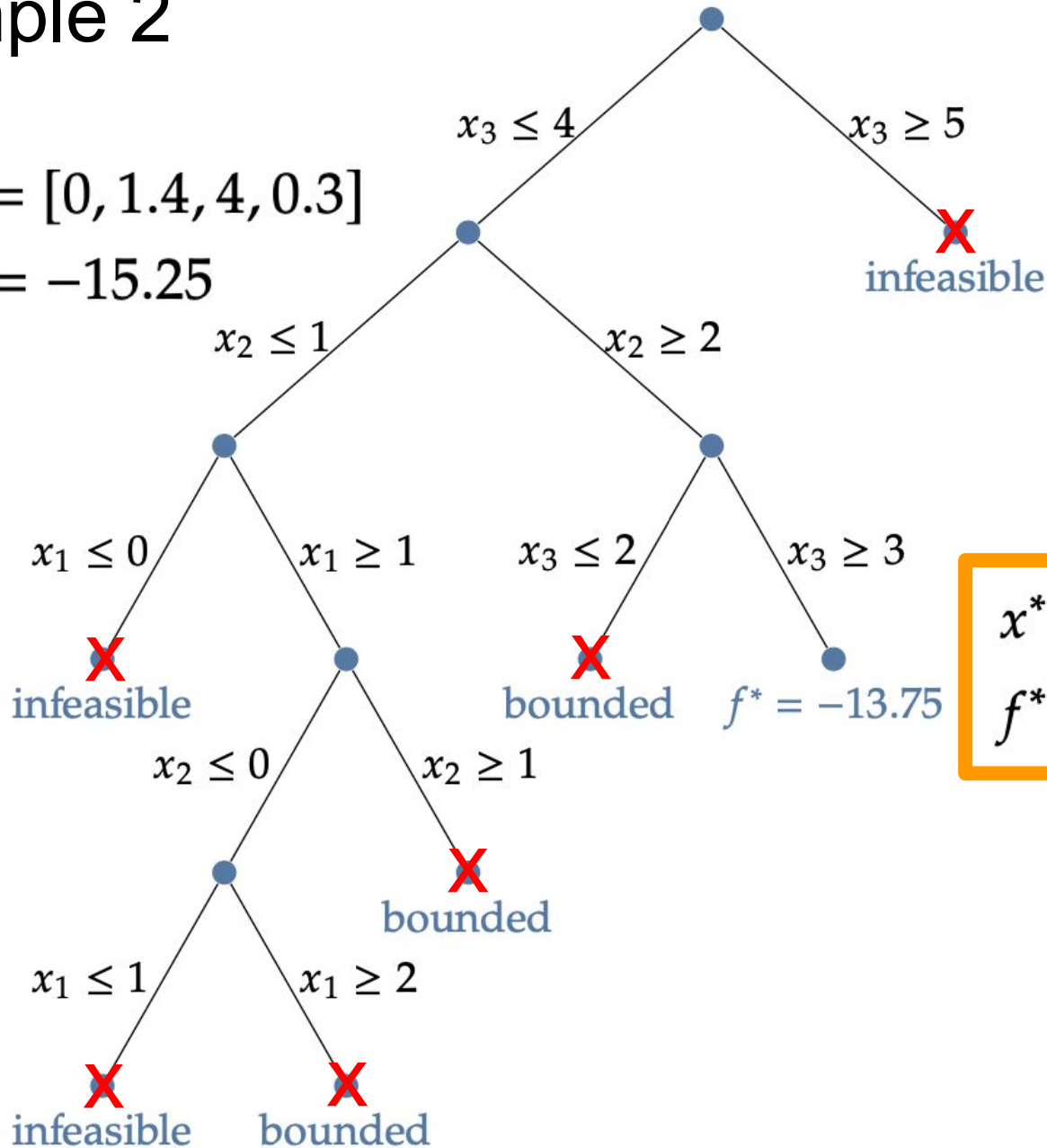


$$x^* = [0, 1.1818, 4.4091, 0], \quad f^* = -15.59$$

Example 2

$$x^* = [0, 1.4, 4, 0.3]$$

$$f^* = -15.25$$



$$x^* = [0, 2, 3, 0.5]$$

$$f^* = -13.75.$$

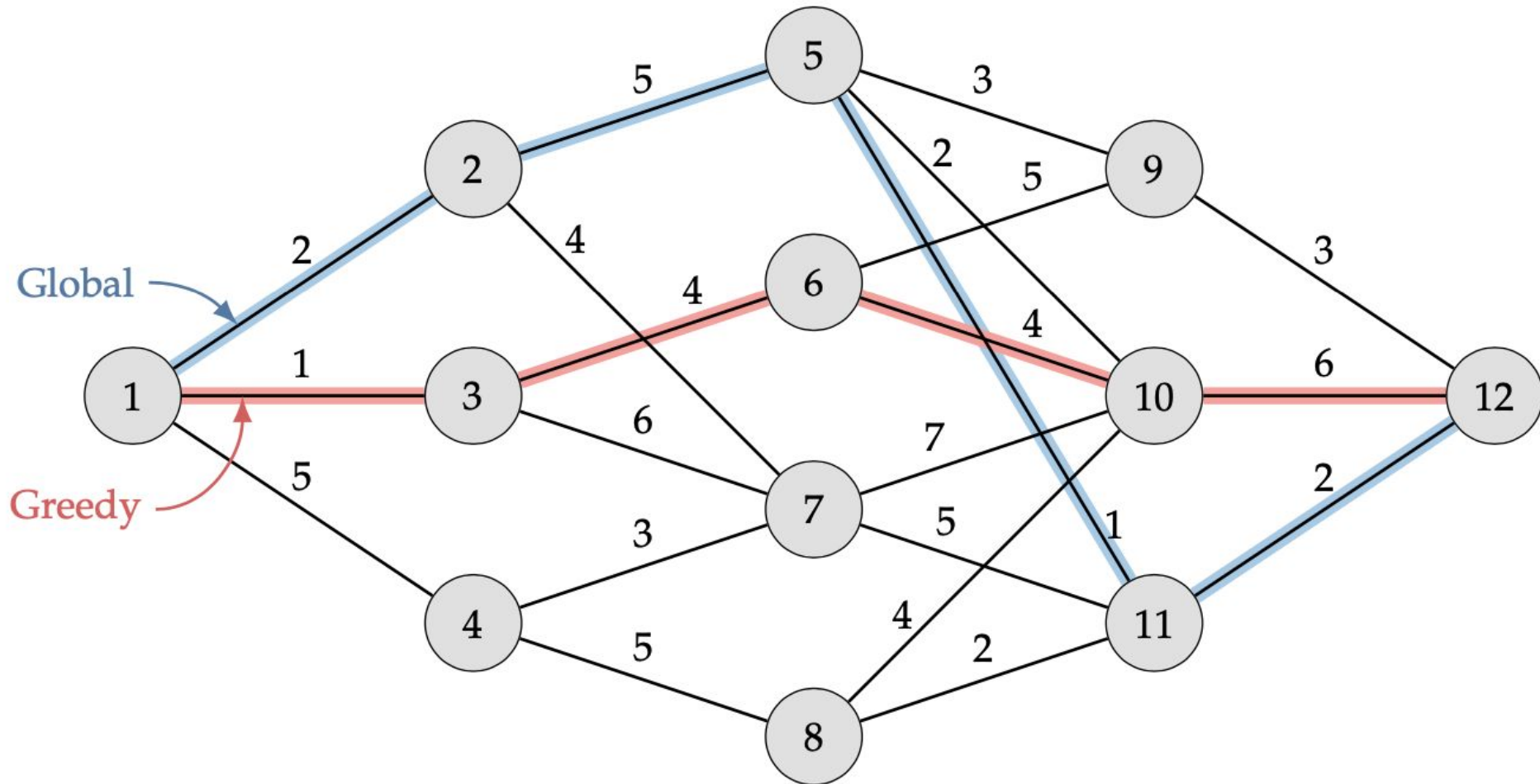
Greedy Algorithms

Greedy Algorithms

- Reduce the problem to a subset of smaller problems (often down to a single choice) and then make a locally optimal decision.
- That decision is locked in, and then the next small decision is made in the same way.
- A greedy algorithm does not revisit past decisions and thus ignores much of the coupling between design variables.

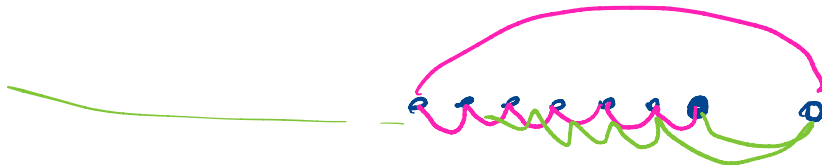
Greedy Algorithms Example

- Which path to take to achieve the lowest possible total cost?



Greedy Algorithms More Examples

- Traveling salesperson problem - always select the nearest city as the next step.
- Optimising a propeller design (number of blades, type of material, number of shear webs etc.) - optimise the variables one at a time, with the others fixed, i.e., optimise the number of blades first, fix that number, then optimise material, and so on.
- Grocery store shopping problem - always pick the cheapest food item next, or always pick the most nutritious food item next, or always pick the food item with the most nutrition per unit cost.



Greedy Algorithms

- Advantages:
 - Algorithms are easy to construct
 - Scalable
 - Can sometimes quickly find solutions reasonably close to an optimal solution.
 - Bound the computational expense of the problem.
- Disadvantages:
 - It usually does not find an optimal solution (and in some cases finds the worst solution! (Gutin et al., 2002)).
 - The solution may also not be feasible.

Gutin et al., “Traveling salesman should not be greedy: domination analysis of greedy-type heuristics for the TSP”, 2002.

Exercises

- Think of an assignment problem that might be useful for you in real life. Solve it using the Hungarian Algorithm.
- Think of an optimisation problem and apply the greedy algorithm to it. How would it proceed?
- Look up the terms that are new to you in this lecture and understand them.

Thanks!

